

Understanding MCP Servers and Protocols: A Beginner-Friendly Exploration

Introduction

Artificial Intelligence (AI) models like ChatGPT are powerful, but they have limits. They can reason, understand, and generate human-like text, but they don't automatically have access to **real-time data** or **external tools**. This is where **MCP servers** (Model Context Protocol servers) come in.

In this blog post and research-style recap, we will explore what MCP servers are, how they differ from HTTP, and how they help Large Language Models (LLMs) become more powerful.

Part 1: The Role of HTTP in Traditional Applications

Before diving into MCP, let's start with something familiar — **HTTP**.

- When you open a web application in your browser and click a button or type in a text box, your request is sent to a **server** using HTTP.
- The server processes the request and sends back a response, often in the form of HTML, CSS, and JavaScript.
- Your browser renders that response into a web page you can see and interact with.

So in short: **HTTP is the communication protocol between your browser (client) and the web server.**

Part 2: Where LLMs Fit In

Now imagine you build an application called *LetsDoAI*. You: - Visit the app using your browser (HTTP request). - Type a question (e.g., *"What's the weather in Paris?"*). - Your app server sends that text to an LLM like ChatGPT using an **HTTP API** request. - The LLM responds with an answer.

If the LLM already knows the answer (from training data), it replies directly.

But here's the problem: **LLMs cannot know real-time information (like today's weather) on their own.**

Part 3: Enter MCP Servers

When the LLM doesn't have enough information, it needs help. This is where **MCP servers** come in.

- The LLM realizes: *"I don't know the weather right now, but I have access to a weather tool via MCP."*

- The LLM sends a request to the **MCP server** using the **MCP protocol**.
- The MCP server queries the weather API and returns structured data (e.g., `{ "temperature": "21C", "condition": "Sunny" }`).
- The LLM takes this result and translates it into a natural language answer for the user.

So the flow looks like this:

1. Client (Browser) → HTTP → Your App Server
2. App Server → HTTP → LLM API (e.g., ChatGPT)
3. LLM → MCP Protocol → MCP Server (tools/data)
4. MCP Server → returns structured result
5. LLM → natural language response → HTTP → Client (Browser)

Part 4: HTTP vs MCP

Here's a direct comparison:

Aspect	HTTP	MCP
Purpose	Communication between clients (browsers/apps) and servers	Communication between LLMs and external tools/ data
Request Type	Web pages, APIs, files	Function calls, tool requests
Who Uses It?	Users, apps, browsers	AI models (LLMs)
Example	<code>GET /weather</code> → returns HTML/JSON	<code>weather.getForecast(city=Paris)</code> → returns structured data

In essence: - **HTTP = the web's universal protocol for clients ↔ servers.** - **MCP = the AI's universal protocol for LLMs ↔ tools.**

Part 5: Do You Always Need MCP?

Not always: - If you are only calling a **public LLM API** (like ChatGPT), you only use **HTTP**. - If you want the LLM to access **extra tools/data** (databases, APIs, functions), then inside the LLM environment, **MCP is used** to connect to those tools.

From the developer's perspective: - You work with **HTTP** when integrating with the LLM API. - The LLM itself may use **MCP** behind the scenes if tools are connected.

Conclusion

To summarize: - **HTTP** connects your app with the LLM. - **MCP** connects the LLM with tools it needs for real-time or external data. - You (as a developer or user) usually only see HTTP directly, unless you build MCP tools yourself.

So, in one sentence: **If the LLM is enough, only HTTP is used; if the LLM needs extra help, it calls MCP tools via the MCP protocol.**

Research-Style Recap

- **Problem:** LLMs lack real-time/world knowledge beyond their training data.
 - **Solution:** MCP servers provide structured access to external tools and data.
 - **HTTP vs MCP:**
 - HTTP = user ↔ app/LLM communication.
 - MCP = LLM ↔ tool communication.
 - **Example:** Asking for Paris weather → LLM calls MCP weather server → MCP fetches data → LLM replies naturally.
 - **Impact:** MCP makes LLMs extensible and practical for real-world applications.
-

Final Thought

Think of it this way: **HTTP is how humans talk to servers, MCP is how LLMs talk to tools.**